

Mock coding tests

COMPLETE LEARNING GUIDE

What You'll Learn

Everything covered so far in the DSA modules is raw technique — this submodule is about performance under interview conditions, where a correct algorithm poorly communicated, or a good idea poorly time-managed, can score worse than a mediocre solution presented well. This guide covers the standard structured approach top interview coaches and companies expect (clarify, examples, brute force, optimize, code, test), concrete time-management strategies for a typical 30-45 minute coding round, how to think aloud effectively without narrating every keystroke, the common question patterns seen across LeetCode-style platforms and what signals they usually test for, a disciplined approach to testing your own code against edge cases before declaring it done, and how to structure a post-interview practice loop that actually improves performance over time rather than just accumulating solved-problem count.

1. The Standard Structured Approach

A strong interview answer follows a recognizable arc that interviewers are trained to look for and reward, precisely because it demonstrates process, not just a correct final answer arrived at by luck or memorization: clarify the problem (restate it, ask about edge cases and constraints), work through one or two concrete examples by hand, state a brute-force solution and its complexity even if you already see a better one, propose and justify an optimization, write the code, then test it against your own examples. Skipping straight to "optimal" code without narrating this arc denies the interviewer the signal they're actually there to collect — they're evaluating your process as much as your answer.

2. Clarifying Questions Before Writing Any Code

A surprising fraction of interview problems are deliberately underspecified, and asking the right clarifying questions before coding is itself part of the evaluation — it signals you won't silently make risky assumptions in a real engineering setting either. Worth asking explicitly: can the input be empty or null? Are there duplicate values, and if so, how should they be handled? What's the expected scale of input size (this affects whether $O(n^2)$ is even acceptable)? Should the solution modify the input in place or return a new structure? Getting these answered up front avoids building a solution to the wrong problem and having to restart under time pressure.

3. Brute Force First, Then Optimize — Out Loud

Stating a correct brute-force solution and its complexity before attempting to optimize serves two purposes: it guarantees you have a fallback if the optimization doesn't pan out in time, and it gives the interviewer a concrete baseline to measure your optimization against, making the improvement legible rather than assumed. When proposing an optimization, name the specific technique driving it explicitly ("this looks like it wants a sliding window because we're computing a running aggregate over a contiguous subrange") — naming the pattern demonstrates recognition, not just a lucky guess, and is a strong signal in itself.

4. Time Management in a 30-45 Minute Round

A rough allocation that works well for a typical single-question round: 3-5 minutes clarifying and working examples, 5-10 minutes discussing approach and complexity before any code, 15-20 minutes writing code, and 5-10 minutes testing and refining — leaving a buffer is more valuable than rushing to "finish" code that hasn't been tested. If you're significantly behind this pacing partway through (still debating approach at the 20-minute mark), it's better to explicitly say so and commit to the best approach discussed so far than to silently keep exploring and run out of time with nothing coded at all.

5. Thinking Aloud Without Over-Narrating

Interviewers need to hear your reasoning to evaluate it, but narrating literally every line as you type it ("now I'm creating a variable called `i` and setting it to zero") is noise that obscures the actual decision-making they're listening for. The useful level of narration is decisions and reasoning, not mechanics: state what data structure you're choosing and why, flag a tricky edge case before you handle it, and briefly explain any non-obvious line, but let straightforward, self-explanatory code be typed largely in silence — silence while typing familiar code is completely normal and expected, not a red flag.

6. Common Question Patterns on LeetCode-Style Platforms

Most interview questions, despite superficially different phrasing, are drawn from a fairly small set of recognizable patterns covered across this course's DSA modules — recognizing which pattern a new problem matches is often 80% of the work. Two Sum-style problems signal hash map lookup or two-pointer (depending on whether the input is sorted); "smallest/largest window satisfying a condition" signals sliding window; "next greater/smaller element" signals a monotonic stack; problems about counting paths or ways signals DP; problems about connected components or reachability signal BFS/DFS; problems about scheduling or intervals often signal greedy (with a DP fallback if greedy provably fails).

7. Testing Your Own Code Before Declaring It Done

Never declare a solution finished immediately after writing the last line — dry-run it against the example from the problem statement first, then deliberately test edge cases: an empty input, a single-element input, all-duplicate values, and the smallest/largest values the constraints allow. This step is exactly what separates candidates who write code that merely compiles from those who write code that's actually correct, and interviewers explicitly notice (positively) when a candidate proactively tests their own solution without being prompted to.

JAVA

```
// After writing a solution, deliberately walk through edge cases before saying  
// "done"  
// Empty input: what does the function do with an empty array/string?  
// - Single element: does a two-pointer loop's "left < right" condition  
// correctly skip work?  
// - Boundary values: smallest/largest per the stated constraints
```

8. A Practice Loop That Actually Improves Performance

Solving a large raw count of problems is far less effective than a deliberate loop: attempt a problem under realistic time pressure, and if stuck past a reasonable point, look at the pattern (not the full solution) and retry; after solving (with or without help), explicitly note which pattern it was and why you did or didn't recognize it quickly; periodically revisit problems from a pattern category you previously struggled with to confirm recognition has actually improved, not just memorized a specific problem's exact solution. Tracking patterns rather than problem counts is what transfers to genuinely novel questions in a real interview, where the exact problem you practiced almost never appears verbatim.

Common Pitfalls

- Jumping straight into coding without clarifying ambiguous requirements, then having to backtrack and restart once a wrong assumption surfaces mid-solution.
- Skipping the brute-force statement entirely and jumping to a half-formed "optimal" idea, leaving no fallback and no baseline for the interviewer to measure improvement against.
- Narrating every keystroke instead of key decisions, burying the reasoning the interviewer actually needs to hear under mechanical noise.
- Declaring a solution "done" immediately after it compiles, without dry-running it against the example or testing edge cases.
- Losing track of time by over-exploring multiple approaches without committing, running out of time with no working code at all.
- Memorizing specific problems' exact solutions instead of the underlying pattern, then failing to recognize the same pattern in a differently-worded but structurally identical new problem.
- Going silent for long stretches without any communication, leaving the interviewer unable to assess reasoning even if the final code turns out correct.

Key Takeaways

- Follow the structured arc: clarify, examples, brute force with stated complexity, optimize, code, test — skipping stages costs more than it saves.
- Ask clarifying questions before coding — underspecified problems are often deliberate, and asking signals sound engineering judgment.
- Name the pattern you're applying explicitly ("this is a sliding window because...") — recognition, stated out loud, is itself a strong signal.

- Budget time explicitly across a round (clarify, discuss, code, test) and communicate if you're falling behind rather than silently running out the clock.
- Always test your own code against the given example plus edge cases (empty, single-element, duplicates, boundary values) before calling it done.
- Practice by pattern recognition, not raw problem count — the goal is transferring technique to novel problems, not memorizing specific solutions.

Learn More

GeeksforGeeks — How to Prepare for Coding Interviews

<https://www.geeksforgeeks.org/dsa/how-to-prepare-for-coding-interviews/>

GeeksforGeeks — Must-Do Coding Interview Question Patterns

<https://www.geeksforgeeks.org/dsa/must-do-coding-questions-for-companies-like-amazon-microsoft-adobe/>

LeetCode — Explore Cards (Pattern-Organized Practice)

<https://leetcode.com/explore/>